Московский государственный технический университет
имени Н. Э. Баумана
Кафедра ИУ5

Лабораторная работа 1

Основные конструкции языка Python

Дисциплина
Парадигмы и конструкции языков программирования

Выполнил: студент группы ИУ5-31Б
Летуновский Артемий Алексеевич

Проверил: Гапанюк Юрий Евгеньевич

Москва 2026

Цель и задание

Цель работы - изучить основные конструкции языка Python на примере решения биквадратного уравнения в процедурной, объектно-ориентированной и функциональной парадигмах.

Необходимо создать три консольные программы для уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$. Каждая программа принимает коэффициенты с клавиатуры или из командной строки, вычисляет дискриминант и действительные корни. Неверный коэффициент запрашивается повторно до корректного ввода.

Алгоритм решения

При $A \neq 0$ вводится замена $t = x^2$. Получаем $A \cdot t^2 + B \cdot t + C = 0$ с дискриминантом $D = B^2 - 4AC$. При $D < 0$ решений нет; при $D = 0$ получаем $t = -B/(2A)$; при $D > 0$ получаем $t_{1,2} = (-B \pm \sqrt{D})/(2A)$.

Каждое $t > 0$ дает два действительных корня $x = \pm\sqrt{t}$. Значение $t = 0$ дает один корень $x = 0$, а $t < 0$ не дает действительных корней. Повторы удаляются, корни выводятся по возрастанию.

При $A = 0$ решается $B \cdot x^2 + C = 0$. Если $B \neq 0$, используется $t = -C/B$. Если $A = B = 0$ и $C \neq 0$, корней нет. Если все коэффициенты равны нулю, решением является любое действительное число.

Структура программы

Файл common.py содержит общие функции ввода и вывода. procedural.py решает задачу функцией solve. В oop.py коэффициенты и методы решения объединены в классе BiquadraticEquation. В functional.py используются чистые функции, map и сопоставление с образцом match/case. Ввод и вывод отделены от вычислений.

None обозначает бесконечное множество решений; пустой список - отсутствие корней. Требуется Python 3.10 или новее. Для обычных учебных коэффициентов используются стандартные вычисления float.

Общие функции ввода и вывода

```
"""Общие функции ввода и вывода для трех программ."""

import math
import sys


def read_coefficients():
    values = []
    for index, name in enumerate(("A", "B", "C"), start=1):
        text = sys.argv[index] if index < len(sys.argv) else None
        while True:
            if text is None:
                text = input(f"Введите {name}: ")
            try:
                value = float(text)
                if not math.isfinite(value):
                    raise ValueError
                values.append(value)
                break
            except ValueError:
                print(f"Ошибка: {name} должен быть конечным действительным числом.")
                text = None
    return tuple(values)


def print_result(a, b, c, result):
    print(f"Уравнение: ({a:g})*x^4 + ({b:g})*x^2 + ({c:g}) = 0")
    print(f"Дискриминант D = {b * b - 4 * a * c:g}")
    if result is None:
        print("Любое действительное число является корнем.")
    elif not result:
        print("Действительных корней нет.")
    else:
        print("Действительные корни:", "; ".join(f"{x:g}" for x in result))
```

Процедурная программа

```
"""Процедурное решение биквадратного уравнения."""

from math import sqrt
from common import read_coefficients, print_result


def solve(a, b, c):
    # Замена t = x^2 сводит уравнение к квадратному.
    if a == 0:
        if b == 0:
            return None if c == 0 else []
        t_values = [-c / b]
    else:
        d = b * b - 4 * a * c
        if d < 0:
            return []
        if d == 0:
            t_values = [-b / (2 * a)]
        else:
            t_values = [(-b - sqrt(d)) / (2 * a),
                        (-b + sqrt(d)) / (2 * a)]

    roots = set()
    for t in t_values:
        if t == 0:
            roots.add(0.0)
        elif t > 0:
            roots.add(-sqrt(t))
            roots.add(sqrt(t))
    return sorted(roots)


if __name__ == "__main__":
    a, b, c = read_coefficients()
    print_result(a, b, c, solve(a, b, c))
```

Объектно ориентированная программа

```
"""Объектно-ориентированное решение биквадратного уравнения."""

from math import sqrt
from common import read_coefficients, print_result


class BiquadraticEquation:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def discriminant(self):
        return self.b * self.b - 4 * self.a * self.c

    def solve(self):
        a, b, c = self.a, self.b, self.c
        if a == 0:
            if b == 0:
                return None if c == 0 else []
            t_values = [-c / b]
        else:
            d = self.discriminant()
            if d < 0:
                return []
            if d == 0:
                t_values = [-b / (2 * a)]
            else:
                t_values = [(-b - sqrt(d)) / (2 * a),
                            (-b + sqrt(d)) / (2 * a)]

        roots = set()
        for t in t_values:
            if t == 0:
                roots.add(0.0)
            elif t > 0:
                roots.add(-sqrt(t))
                roots.add(sqrt(t))
        return sorted(roots)


if __name__ == "__main__":
    equation = BiquadraticEquation(*read_coefficients())
    print_result(equation.a, equation.b, equation.c, equation.solve())
```

Функциональная программа

```
"""Функциональное решение с сопоставлением с образцом."""

from math import sqrt
from common import read_coefficients, print_result


def roots_from_t(t):
    match t:
        case 0:
            return (0.0,)
        case value if value > 0:
            return (-sqrt(value), sqrt(value))
        case _:
            return ()


def solve(a, b, c):
    # Чистая функция: результат зависит только от коэффициентов.
    match (a, b, c):
        case (0, 0, 0):
            return None
        case (0, 0, _):
            return []
        case (0, _, _):
            t_values = (-c / b,)
        case _:
            d = b * b - 4 * a * c
            match d:
                case value if value < 0:
                    return []
                case 0:
                    t_values = (-b / (2 * a),)
                case _:
                    t_values = ((-b - sqrt(d)) / (2 * a),
                                (-b + sqrt(d)) / (2 * a))

    # map преобразует корни t в корни x, set убирает повторы.
    return sorted({x for group in map(roots_from_t, t_values)
                   for x in group})


if __name__ == "__main__":
    a, b, c = read_coefficients()
    print_result(a, b, c, solve(a, b, c))
```

Примеры выполнения программы

Пример 1 Четыре действительных корня

```
> python procedural.py 1 -5 4
Уравнение: (1)*x^4 + (-5)*x^2 + (4) = 0
Дискриминант D = 9
Действительные корни: -2; -1; 1; 2
```

Пример 2 Повторный корень вспомогательного уравнения

```
> python oop.py 1 -2 1
Уравнение: (1)*x^4 + (-2)*x^2 + (1) = 0
Дискриминант D = 0
Действительные корни: -1; 1
```

Пример 3 Отрицательный дискриминант

```
> python functional.py 1 0 1
Уравнение: (1)*x^4 + (0)*x^2 + (1) = 0
Дискриминант D = -4
Действительных корней нет.
```

Пример 4 Положительный D и отрицательные t

```
> python functional.py 1 3 2
Уравнение: (1)*x^4 + (3)*x^2 + (2) = 0
Дискриминант D = 1
Действительных корней нет.
```

Примеры ввода и особых случаев

Пример 5 Ошибочный аргумент и повторный ввод

```
> python procedural.py bad -5 4
Ошибка: A должен быть конечным действительным числом.
Введите A: wrong
Ошибка: A должен быть конечным действительным числом.
Введите A: 1
Уравнение: (1)*x^4 + (-5)*x^2 + (4) = 0
Дискриминант D = 9
Действительные корни: -2; -1; 1; 2
```

Пример 6 Ввод с клавиатуры

```
> python oop.py
Введите A: 1
Введите B: -5
Введите C: 4
Уравнение: (1)*x^4 + (-5)*x^2 + (4) = 0
Дискриминант D = 9
Действительные корни: -2; -1; 1; 2
```

Пример 7 Вырожденное уравнение

```
> python functional.py 0 1 -4
Уравнение: (0)*x^4 + (1)*x^2 + (-4) = 0
Дискриминант D = 1
Действительные корни: -2; 2
```

Пример 8 Бесконечное множество решений

```
> python functional.py 0 0 0
Уравнение: (0)*x^4 + (0)*x^2 + (0) = 0
Дискриминант D = 0
Любое действительное число является корнем.
```

Проверка и вывод

Для каждой из трех программ проверены 14 наборов коэффициентов: четыре корня, повторный корень по t, отрицательный дискриминант, отрицательные значения t, нулевой корень, вырожденные уравнения, отрицательное A и дробные коэффициенты. Полученные корни дополнительно подставлены в исходное уравнение.

Выполнены 18 проверок консольного ввода: полный набор аргументов, ввод с клавиатуры, повторные ошибки, частичный набор аргументов и отклонение значений nan и inf. Все проверки пройдены.

Порядок запуска

Все четыре файла Python должны лежать в одной папке. Из корня репозитория можно выполнить следующие команды:

```
python lab1/procedural.py 1 -5 4
```

```
python lab1/oop.py 1 -5 4
```

```
python lab1/functional.py 1 -5 4
```

Для ввода с клавиатуры команда запускается без коэффициентов. Дробные значения записываются через точку. Если передана только часть коэффициентов, оставшиеся запрашиваются с клавиатуры.

Вывод

Разработаны три программы, решающие одно биквадратное уравнение различными способами. Процедурный вариант организован через функции, объектно-ориентированный - через класс с данными и методами, функциональный - через чистые функции и сопоставление с образцом. Во всех вариантах реализованы проверка ввода и вывод действительных корней.

Источник задания

https://github.com/ugapanyuk/BKIT_2022/wiki/lab_python_intro

Репозиторий работы: https://github.com/artem20080205-bit/LabaASOIU